

Hindi→English Crosslingual Fact-Check Retrieval

SemEval-2025 Task 7 — Detailed Technical Report

Prepared from project notebook: `Hineng.ipynb`

September 2026

Contents

Hindi→English Crosslingual Fact-Check Retrieval — Detailed Technical Report	1
1. Purpose and Scope	1
2. Dataset and Task Definition	2
3. Text Representation	4
4. Retrieval Model and Encoding	4
5. Fine-Tuning Method — QLoRA Contrastive Learning	5
6. Test-Set Evaluation	7
7. Why Scores Sometimes Increased and Sometimes Didn't — Summary of Causes	10
8. Overall Assessment	11

Hindi→English Crosslingual Fact-Check Retrieval — Detailed Technical Report

1. Purpose and Scope

This report documents, in full detail, the Hindi (post) → English (fact-check claim) crosslingual retrieval pipeline built in `Hineng.ipynb`, part of the SemEval-2025 Task 7 (“Multilingual and Crosslingual Fact-Checked Claim Retrieval”) crosstrack. It covers every numerical figure produced by the notebook, the method behind each stage, the resulting scores, and — most importantly — **why** particular scores moved the way they did across baseline, fine-tuning, and ensemble stages.

The task: given a social-media post written in Hindi, retrieve the correct previously fact-checked claim (written in English) from a large multilingual pool of candidate claims. This is a **crosslingual dense retrieval** problem — the query language (Hindi) differs from the target document language (English), and the candidate pool is *not* restricted to English claims; it contains all languages present in the shared task’s crosslingual subset.

2. Dataset and Task Definition

2.1 Source data

The notebook uses the official SemEval-2025 Task 7 data (`semeval2025_task7_data.zip`), containing:

- `fact_checks.csv` — the full multilingual pool of fact-checked claims (153,743 rows in train/dev, 272,447 rows in test)
- `posts.csv` — social media posts across many languages (24,431 rows in train/dev, 8,276 rows in test)
- `pairs.csv` / `pairs_dev_crosslingual.csv` / `pairs_test_crosslingual.csv` — ground-truth post $\leftrightarrow$ claim links
- `tasks.json` / `crosslingual_reference.json` — the official task splits and test-set answer key

2.2 Crosslingual pool definition

The crosslingual subtask (as opposed to monolingual) pools **all 52 language combinations** together — it is *not* split by language pair in the source task files. This means Hindi $\rightarrow$ English is not a pre-packaged subset; it had to be carved out manually from the general crosslingual pool.

Numbers verified against the paper (Table 2), all matched exactly via hard assert statements in the code:

Quantity	Notebook Value	Paper Expects	Match
Fact-checks in crosslingual pool	153,743	153,743	exact match
Posts (train)	4,972	4,972	exact match
Posts (dev)	552	552	exact match
Crosslingual train pairs (all languages)	5,787	5,787	exact match
Crosslingual dev pairs (all languages)	651	651	exact match

This step matters because it confirms the notebook is working from the *correct, unmodified* version of the official dataset before any Hindi-specific filtering happens — ruling out a whole category of “wrong data” errors before the real work starts.

2.3 Hindi-post extraction (Cell 5)

Because the crosslingual pool isn’t pre-split by language, a **language detector** had to be built to pull out Hindi-authored posts from the $4,972 + 552 = 5,524$ crosslingual train+dev posts. The detector logic, in priority order:

1. **Confident language code from the post’s text field** — the dataset stores each post’s original text plus machine-translation metadata as a tuple (original_text, translated_text, [(lang_code, probability), ...]). The detector takes the *highest-probability* language code from that list.
2. **OCR fallback** — if the text field itself doesn’t yield a code, the same lookup is applied to the OCR field (text extracted from any attached image), since some posts are image-only with a caption.
3. **Devanagari script fallback** — only if *no* language code is available at all (neither from text nor OCR) does the detector fall back to a regex check for Devanagari Unicode characters (\u0900–\u097F) in the *original, untranslated* text.
4. **Explicit exclusion of hi-Latn** — the language-ID library used by the dataset’s creators tags romanized Hindi (Hindi written in Latin script, e.g. “kya baat hai”) as hi-Latn. This tag was deliberately **excluded** from the “confident Hindi” set because manual inspection showed it firing on false positives (e.g. an unrelated post about “Dubai Irani market, Ajman” was tagged hi-Latn). Including it would have inflated the Hindi set with non-Hindi, Latin-script text.

Why this multi-tier approach matters: a naive approach (e.g., “does the text contain Devanagari characters”) would miss posts that are pure OCR (image-only) and would also fail to distinguish real Hindi from romanized Hindi. The tiered fallback maximizes precision (trusting an explicit language code first) while still catching edge cases (script fallback only when no code exists).

Result:

Split	Notebook count	Paper Table 3 expects	Gap
Hindi posts detected (train+dev pool)	756	—	—
hin→eng train pairs	854	—	—
hin→eng dev pairs	99	—	—
Combined train+dev	953	964	11 pairs (1.1%)
hin→eng test pairs	2,321	2,337	16 pairs (0.7%)

Why the gap exists, and why it’s acceptable: the notebook does not silently absorb this discrepancy — it runs a documented investigation. It checked whether the missing ~11 pairs could be recovered by loosening the detector (e.g., re-including hi-Latn), and found that doing so would *also* pull in posts from roughly ten *other* languages that happen to pair exclusively with English claims (Chinese, Sinhala, Urdu, Korean, Tagalog, etc.) — meaning there is no clean way to force the count to exactly 964 without contaminating the Hindi set with non-Hindi posts. Given that a 1.1% (train+dev) and 0.7% (test) shortfall is small and the *reason* for it is understood and documented, the notebook proceeds on the detected set (953 / 2,321) rather than chasing an exact match that would require compromising precision elsewhere. A TOLERANCE band (15 pairs for train+dev, 40 for test) is hard-coded and asserted against, so if a future re-run produced a much larger, unexplained gap, the notebook would fail loudly rather than silently continuing on bad data.

(Minor note: one code comment describes this as “the same category of discrepancy” as a separate ~22% eng-eng test-set gap noted elsewhere in the broader project.

That comparison overstates the similarity — 1.1% and 22% are very different magnitudes of missing data, even if the root cause (dataset-version/language-detection ambiguity) is arguably similar in kind.)**

2.4 Claim pool stays unrestricted

Critically — and this is what makes the task “crosslingual” rather than “Hindi-to-English-only” — the candidate pool for retrieval is **not** filtered down to English claims. All 153,743 (train/dev) or 272,447 (test) claims, across every language in the dataset, remain in the search space for every Hindi query. A sanity check in Cell 7 confirms this explicitly (`assert len(cross_claims_pool_df) == 153743`). This is harder than a monolingual or filtered-pool setup: the model has to correctly rank the one true English claim above ~153,000+ distractor claims in *other* languages too, not just above other English claims.

3. Text Representation

3.1 Fused text (post text + OCR)

Many posts are short or vague captions where the actual claim content lives in an attached image, extracted via OCR. To avoid losing that signal, the notebook builds a **fused text** field per post:

- If only main text exists → use main text.
- If only OCR exists → use OCR text.
- If both exist and OCR text is already contained in the main text → use main text only (avoid duplication).
- If both exist and are meaningfully different → concatenate as "{main text} [OCR]: {ocr text}".

Result: 756/756 Hindi posts (100%) yielded usable fused text; 153,743/153,743 claim texts (100%) were usable. No pairs were dropped due to empty or malformed text (Cell 9: “Dropped (empty/bad text): 0.00%”).

Why this matters for scores: if fusion hadn’t been applied, any post that is only an image (no caption, or a vague one like “look at this!”) would have an empty or uninformative query embedding, and the true claim would essentially be unretrievable no matter how good the encoder is. Fusion recovers signal that would otherwise be a hard, unfixable ceiling on recall.

4. Retrieval Model and Encoding

4.1 Base encoder

- **Model:** `intfloat/multilingual-e5-large` — a 24-layer XLM-R-based encoder (560M params) pretrained specifically for multilingual dense retrieval, using contrastive pretraining on multilingual text pairs.

- **Pooling:** mean pooling over token embeddings, masked by the attention mask (standard for E5-family models — *not* CLS-token pooling).
- **Normalization:** L2-normalized embeddings, so cosine similarity = dot product.
- **Required text prefixes:** E5 models are trained with instructional prefixes — "query: " for queries (Hindi posts) and "passage: " for documents (claims). The notebook correctly applies these throughout; getting this wrong is a common and severe source of degraded E5 performance, and it was handled correctly here.
- **Max sequence length:** 128 tokens.
- **Precision:** fp16 for both training and inference forward passes (with `torch.autocast`).

4.2 Hard negative mining (Cell 10-11)

Before fine-tuning, the **frozen** base encoder is used to embed: - All 153,743 pool claims (took ~8.5 minutes on a Tesla T4 GPU, 2,403 batches at batch size 64). - All 677 Hindi training posts.

These embeddings are cached to Google Drive so this expensive step never needs to be redone across sessions.

For each training post, the top-25 most similar claims (by cosine similarity against the *frozen* model) are retrieved, the true positive claim(s) are removed from that list, and the top 5 remaining are kept as **hard negatives** — i.e., claims that look deceptively similar to the query under the *current, untrained* model, but are actually wrong.

Why hard negatives instead of random negatives: random negatives (a random claim from the 153K pool) would almost always be trivially different from the query and provide little training signal — the model would learn nothing beyond “an unrelated claim in a different language looks different.” Hard negatives force the model to learn the *fine-grained* distinctions that actually matter for this task: two claims that use similar wording/topic but represent different underlying facts.

4.3 Training example construction

854 usable training examples were built (out of 854 raw train pairs — 100% survival, since all texts were valid and every post had at least one valid hard negative). Each example = {query: <Hindi post fused text>, positive: <true English claim text>, negatives: [5 hard negative claim texts]}.

5. Fine-Tuning Method — QLoRA Contrastive Learning

5.1 Why QLoRA

Fully fine-tuning a 560M-parameter encoder on a Tesla T4 (16 GB VRAM, a modest/free-tier Colab GPU) is impractical at full precision. QLoRA solves this by: 1. Loading the base model in **4-bit NF4 quantization** (bitsandbytes), cutting memory footprint roughly 4x versus fp16/fp32. 2. Injecting small trainable **LoRA adapters**

(rank 8, alpha 16) into the query, key, and value projection matrices of the attention layers, while the quantized base weights stay frozen.

Trainable parameters: 1,179,648 out of 561,070,080 total — **0.21%** of the model. This is what makes iterative fine-tuning experiments feasible on a single free GPU within a Colab session’s time limits.

5.2 Training configuration

Hyperparameter	Value
LoRA rank (r)	8
LoRA alpha	16
LoRA target modules	query, key, value
LoRA dropout	0.1
Batch size	16
Gradient accumulation	2 (effective batch = 32)
Epochs	3
Learning rate	3e-5
LR schedule	Linear warmup (10%) + linear decay
Contrastive temperature	0.05
Negatives per query per step	3 (sampled from the 5 mined hard negatives)
Training examples	854
Steps per epoch	54 (854 / 16 $\approx$ 54)

5.3 Loss function

A standard **InfoNCE-style contrastive loss** is used per training step: 1. Encode the batch of queries, positives, and negatives (with query: / passage: prefixes respectively). 2. Compute cosine similarity between each query and its own positive (pos_sim), and between each query and its 3 sampled negatives (neg_sim). 3. Concatenate [pos_sim, neg_sim] into a single logits vector per example, scale by 1/temperature = 20, and apply cross-entropy loss with the positive always at index 0.

This is exactly the loss formulation used in most modern dense-retrieval fine-tuning setups (e.g., DPR, Contriever, E5’s own training recipe) — it directly optimizes the model to rank the true claim above the hard negatives in the same embedding space used at inference time, which is the ideal alignment between training objective and evaluation metric.

5.4 Training dynamics — dev evaluation per epoch

A quick_dev_eval function ran after every epoch, encoding all 99 Hindi dev posts as queries against the **full 153,743-claim pool** (not a restricted subset — same difficulty as the real evaluation) and computing Success@1/5/10.

Epoch	Success@1	Success@5	Success@10	Training loss (end of epoch)
1	0.1139	0.4684	0.5696	1.499
2	0.1646	0.5190	0.6076	1.426
3	0.1646	0.5190	0.6076	1.387

Why scores increased from epoch 1 → 2: the model had only seen the 854 training examples once (epoch 1); by epoch 2, it has had a second pass to sharpen the separation between hard negatives and true positives that the loss is explicitly optimizing for. Training loss dropping (1.499 → 1.426) alongside dev Success@1 improving (0.114 → 0.165) confirms the model was still usefully learning during this pass.

Why scores did *not* increase from epoch 2 → 3, despite loss continuing to drop (1.426 → 1.387): this is the classic signature of **the model continuing to fit the training distribution (loss keeps falling) without that improvement generalizing to the dev set (dev metrics flatline exactly)**. With only 854 training examples and a 0.21%-parameter adapter, the model reached its useful-generalization ceiling by epoch 2 — the further loss reduction in epoch 3 reflects the model getting *more confident* about training examples it had already learned to separate correctly, not learning anything new that transfers to unseen dev posts. This is a small-data regime, so plateauing after 2 epochs is expected rather than alarming; it does **not** indicate an error in the pipeline, just that 3 epochs was already slightly past the point of diminishing returns for this dataset size. Since the code explicitly checkpoints “best-on-dev-Success@10” and epoch 2’s adapter is what gets carried forward as `lora_best_dev` (epoch 3 didn’t beat it, so it wasn’t overwritten), the final evaluation model is effectively the epoch-2 adapter — training for a 3rd epoch cost extra GPU time without changing the deployed model.

6. Test-Set Evaluation

6.1 Test data

Quantity	Value
Total test fact-checks (full pool)	272,447
Total test posts	8,276
Crosslingual test posts (task-defined pool)	4,000
Hindi-detected test posts	1,637
hin→eng test pairs (usable)	2,321

6.2 Metric — Success@k

`success@k` = (number of queries where the true claim appears
in the top-k retrieved results) / (total number of queries)

This is computed for $k = 1, 5, 10$ by embedding all 1,637 Hindi test posts and all 272,447 test claims, computing the full similarity matrix (in chunks of 128 queries to manage GPU memory), and taking the top-10 highest-similarity claims per post. Success@1 measures “did the model get it exactly right on the first guess”; Success@10 is a much more forgiving “was the right answer anywhere in a short list” measure — useful because in a real fact-checking assistance tool, a human moderator would typically review a short list of candidates rather than trust a single top-1 result blindly.

6.3 Three models compared

(a) Frozen ME5-Large (no fine-tuning) — the baseline:

Metric	Score
Success@1	0.1924
Success@5	0.5113
Success@10	0.6139

This uses the exact same fused-text pipeline and prefixes as the fine-tuned model, so any difference in score is attributable *only* to the fine-tuning, not to a difference in text preprocessing.

(b) Fine-tuned ME5-Large + LoRA (best-on-dev adapter, i.e., epoch 2’s weights):

Metric	Score	Absolute gain vs. frozen	Relative gain
Success@1	0.2260	+0.0336	+17.5%
Success@5	0.5840	+0.0727	+14.2%
Success@10	0.6891	+0.0752	+12.3%

Why every metric improved, and why the improvement is *larger* at Success@1 than at Success@10: fine-tuning with hard negatives specifically teaches the model to break ties between claims that look similar under the frozen embedding space. That kind of fine-grained discrimination has its biggest payoff exactly where fine ranking precision matters most — getting the single best answer to rank *first* (Success@1) — because that’s precisely the failure mode hard-negative training targets (claims that were already “close” under the frozen model, just not close enough to overtake the correct one). By Success@10, the frozen model was *already* getting the right claim into a top-10 list reasonably often (61.4%) simply because a top-10 window is forgiving; fine-tuning still helps push additional cases into that window, but there’s less room left to improve relative to the harder Success@1 case, so the relative gain shrinks as k grows (17.5% $\rightarrow$ 14.2% $\rightarrow$ 12.3%).

(c) Ensemble (frozen + fine-tuned, dev-tuned blend):

The ensemble combines similarity scores as $\text{combo} = \alpha * \text{sims_finetuned} + (1 - \alpha) * \text{sims_frozen}$, where α is chosen entirely on the **dev set** (never

touching test, to avoid leakage) by sweeping $\alpha \in \{0.0, 0.1, \dots, 1.0\}$ and picking the value that maximizes Success@1 subject to Success@5 and Success@10 not decreasing versus the fine-tuned-only baseline.

Dev-set alpha sweep results (abbreviated):

Alpha	Dev Success@1	Dev Success@5	Dev Success@10
0.0 (frozen only)	0.1013	0.3544	0.4810
0.5	0.1013	0.4177	0.5316
0.9	0.1266	0.4810	0.6076
1.0 (fine-tuned only)	0.1392	0.4937	0.6076

Why alpha=1.0 (i.e., no blending at all) was selected as “best”: across the entire sweep, dev Success@1 is *monotonically non-decreasing* as alpha increases from 0 to 1, and the fine-tuned-only model (alpha=1.0) achieves the single highest Success@1 of the whole sweep while tying for the best Success@10. In other words, the frozen model’s similarity scores never added information the fine-tuned model didn’t already have for this dataset — the frozen model is *strictly dominated* by the fine-tuned model on every metric at every blend ratio tested. This makes sense: the fine-tuned model isn’t a *different* model architecture with different strengths (which is what typically makes ensembling useful) — it’s literally the frozen model’s own weights, adjusted via LoRA. Blending in the pre-adjustment weights can only dilute the adjustment, never add complementary information, so a monotonic improvement toward alpha=1.0 is the expected outcome, not a surprising one. On test, this means:

Metric	Ensemble Score	Same as fine-tuned-only?
Success@1	0.2260	[OK] identical
Success@5	0.5840	[OK] identical
Success@10	0.6891	[OK] identical

Mathematically, $\text{combo} = 1.0 * \text{sims_finetuned} + 0.0 * \text{sims_frozen} = \text{sims_finetuned}$ exactly, so no re-computation was even needed on test — the notebook correctly recognized this and reused the fine-tuned results rather than wasting GPU time re-encoding.

6.4 Full comparison table

Model	Success@1	Success@5	Success@10
Frozen ME5-Large (fused text)	0.1924	0.5113	0.6139
Fine-tuned ME5-Large + LoRA (fused text)	0.2260	0.5840	0.6891

Model	Success@1	Success@5	Success@10
Ensemble (fine-tuned + frozen, dev-tuned $\alpha=1.0$)	0.2260	0.5840	0.6891

7. Why Scores Sometimes Increased and Sometimes Didn't — Summary of Causes

Observation	Cause
Dev Success@1/5/10 rose from epoch 1 → epoch 2	Model had a second full pass over the 854 training pairs; hard-negative contrastive loss was still actively separating true positives from confusable negatives; training loss was still falling meaningfully (1.499 → 1.426).
Dev Success@1/5/10 stayed flat from epoch 2 → epoch 3, despite loss still falling (1.426 → 1.387)	Classic small-data plateau: the model kept getting more confident on training examples it already ranked correctly (loss ↓) without learning anything new that generalizes to unseen dev posts (dev metrics unchanged). With 854 examples and a 0.21%-parameter adapter, 2 epochs was already close to the ceiling.
Fine-tuned model beat frozen model on every test metric	Hard-negative contrastive fine-tuning directly targets the failure mode that matters most for this task: distinguishing the true claim from claims that only <i>look</i> similar under the pretrained embedding space.
The <i>relative</i> improvement from fine-tuning shrank as k grew (17.5% at @1 → 12.3% at @10)	Success@10 gives the model 10 chances to be right, so the frozen baseline was already capturing a good share of cases in that wider window; fine-tuning's biggest value-add is in tightening the <i>top</i> of the ranking (Success@1), where the frozen model's errors were concentrated.

Observation	Cause
Ensemble never beat fine-tuned-only, at any blend ratio, and best alpha = 1.0	The frozen and fine-tuned models are not independent estimators — the fine-tuned model <i>is</i> the frozen model plus a learned correction. Blending back in the pre-correction scores can only pull performance toward the (worse) frozen baseline, never add complementary signal, so pure fine-tuned-only was mathematically the best available combination on this dataset.
Hindi-pair counts (953/964, 2,321/2,337) fell slightly short of the paper’s reported numbers	Language detection on a crosslingual dataset spanning 50+ languages is inherently imperfect at the margins; the notebook verified the shortfall isn’t an easy fix (loosening the detector would contaminate the set with other languages), so it proceeded on the documented, smaller-but-correct set rather than forcing an exact match.

8. Overall Assessment

Strengths: - Every dataset-derived number is checked against the paper’s published tables with explicit asserts, not just printed and trusted. - The retrieval pool is genuinely left unrestricted (all languages), which is the methodologically correct way to evaluate a *crosslingual* retrieval claim rather than accidentally simplifying it into an English-only retrieval task. - Hard-negative mining, correct E5 query/passage prefixing, QLoRA efficiency, and dev-only hyperparameter selection (the alpha sweep) are all textbook-correct choices that directly explain why the reported gains are trustworthy rather than artifacts of a leaky evaluation. - The one place fine-tuned and frozen models were compared, they used identical text preprocessing, isolating fine-tuning as the only variable — good experimental hygiene.

Main limitation: - The frozen-model test results in one of the later “resume” cells were manually copied from an earlier printed run’s output rather than reloaded from a saved file, which is the only place in the notebook where a number isn’t independently re-derivable from a saved artifact. - Training data size (854 pairs) inherently caps how much fine-tuning can achieve; the epoch 2→3 plateau reflects that ceiling rather than a flaw in the method.

Bottom line: QLoRA fine-tuning of multilingual-e5-large on hard negatives mined from the frozen model produces a consistent, well-isolated improvement over the frozen baseline on Hindi→English crosslingual claim retrieval (Success@1: 0.192→0.226, Success@5: 0.511→0.584, Success@10: 0.614→0.689), and the dev-tuned ensemble search correctly and honestly concludes that the fine-tuned model

alone is the best available configuration for this language pair and dataset size.